

6.3 Contratos: formulário de criação avulsa — status de entrega

Bloco 6.3 Contratos: formulário de criação avulsa · Sprint 6 — Contratos · Data: 2026-09-23 · Branch: `main` · Ambiente: `gc-dev` (`gzbmhgnpoehoimnidmgg`)

Resumo

Bloco concluído. A listagem ganhou o botão **Novo contrato**, que abre `/contratos/novo`: o caminho manual, para contrato sem proposta de origem. Sem migration nova. `migration list` continua 23/23 alinhadas.

1. As quatro atividades do bloco

ATIVIDADE DO CLICKUP	ESTADO
Seções: identificação (obra, número, descrição), datas (assinatura, prazo_execucao)	<code>contratos/contrato-form.tsx</code> , o mesmo formulário do "Gerar contrato" do 6.2, sem a seção de itens
Valores e percentuais de pagamento com a mesma validação de soma <= 100%	a soma usa <code>validarSomaPctForm</code> de <code>@/lib/propostas</code> , a mesma regra da proposta, com a caixa "Soma das parcelas" ao vivo
Condições de pagamento e observação	seções "Condições de pagamento" (percentuais e campo livre) e "Observações"
Zod espelhando as constraints do banco	<code>contratoSchema</code> em <code>src/lib/contratos-form.ts</code> , com 6 casos de <code>node --test</code> , e <code>validarPayloadContrato</code> repetindo a regra na action

Arquivos: `src/app/(app)/contratos/novo/{page,novo-form}.tsx` e `actions.ts` (`createContrato`), o botão em `contratos/page.tsx`, e `src/lib/contratos-form.ts` (+ `contratos-form.test.ts`).

2. Decisões

2.1 O zod do contrato saiu da pasta da rota e foi para `src/lib/`

No 6.2 ele ficava em `contratos/contrato-form-helpers.ts`, e o documento daquele bloco registrou como pendência que ele não tinha teste unitário. O enunciado do 6.3 pede "zod espelhando as constraints do banco". Para isso ser provado por `node --test`, o schema mudou para

`src/lib/contratos-form.ts`, pelo mesmo motivo de `src/lib/itens-form.ts` (imports relativos, sem React). A pendência do 6.2 sobre `propostaToContratoFormValues` continua: o helper mudou de lugar, mas nenhum caso novo o cobre.

O que o schema espelha:

CONSTRAINT DO BANCO	NO SCHEMA
<code>numero text not null + unique (empresa_id, numero)</code>	obrigatório, até 50 caracteres. A unicidade só o banco sabe, e a action traduz o erro para "Já existe um contrato com esse número"
<code>contratosobra_fk</code>	<code>obra_id</code> tem de ser uuid ("Selecione uma obra")
<code>valor_total >= 0, desconto >= 0, contratos_desconto_valido</code>	<code>validarDesconto</code> , com o erro no campo Desconto
<code>pct_* between 0 and 1</code>	cada percentual entre 0% e 100%
<code>contratos_pct_soma (<= 1)</code>	<code>validarSomaPctForm</code> , com o erro no último percentual

Data de assinatura e prazo são opcionais, como no banco: o contrato pode ser lançado antes de assinar.

2.2 A action repete a regra, sobre o payload

O zod só roda no navegador, e a action é um POST que qualquer um chama. Então `validarPayloadContrato` confere as mesmas regras sobre o payload, com os percentuais já em fração. A camada de escrita manda cada recusa direto na action, sem passar pelo form, e confere que nada foi gravado.

2.3 Campos listados um a um no insert

O insert não faz `...input . status` (sempre `ativo`), `proposta_origem_id` (sempre nulo), `empresa_id` (a do perfil), `created_by`, `anexos` e `historico` não vêm do corpo da requisição. É o erro que o 5.1 corrigiu nos itens (spread depois dos campos do servidor). Há passo mandando `status: 'concluido'`, uma proposta de origem, um histórico e outra empresa no corpo, e o contrato nasce ativo, sem origem e com histórico vazio.

2.4 Depois de criar, a listagem filtrada

Igual ao 6.2: o detalhe é do 6.4, então criar leva a `/contratos?busca=<número>`.

3. Números reais do `npm run validar`

Rodado em 2026-09-23 contra gc-dev, na cópia isolada do working tree (`.next` próprio, portas 3311/9422). Passou na primeira rodada.

CAMADA	ANTES DO 6.3 (FIM DO 6.2)	AGORA	
estático	limpo	<code>tsc --noEmit limpo,</code> <code>warnings or errors</code>	✓ No ESLint ok
unitário	189 casos	195 casos, 0 falhas	+6
build	35 rotas	36 rotas, iguais à baseline	+1 (<code>/contratos/novo</code>)
runtime	76/76 rotas	80/80 rotas ok	+4 (e 3 rotas antigas com trechos novos)
dados	26/26 checagens	27/27 checagens ok	+1
escrita	177/177 passos	191/191 passos ok	+14
navegador	73/73 passos	78/78 passos ok , console limpo	+5

A baseline de rotas foi regravada com `validar.sh build --aceitar-rotas`. O diff foi exatamente uma linha, `/contratos/novo`.

Os 6 casos unitários (`contratos-form.test.ts`): o contrato mínimo válido (sem data e sem prazo) · número obrigatório e até 50, obra obrigatória · desconto maior que o valor recusado, igual aceito, valor e desconto negativos recusados · soma até 100% aceita, 110% recusada no último campo, percentual isolado acima de 100% ou abaixo de 0% · conversão do form para o payload (30% → 0,3, texto vazio → nulo) · `validarPayloadContrato` com as mesmas recusas.

Runtime: 4 rotas novas: o form com as seções (e sem "Itens e origem"), o comercial abre, o visualizador e o financeiro levam 307. Nas antigas, `/contratos` passou a esperar "Novo contrato" para admin e comercial, e o visualizador passou a esperar a ausência do botão.

Dados: a query de obras da page, com o JOIN do cliente vindo como objeto.

Os 14 passos da escrita (13 do fluxo e 1 de limpeza): admin cria · nasce ativo, sem origem, histórico vazio e com o autor · valores, desconto, final (1.800), % e textos gravados · **status, origem, histórico e empresa vindos do corpo são ignorados** · a action recusa, sem gravar: desconto maior que o valor, soma acima de 100%, percentual isolado acima de 100%, valor negativo, número em branco e contrato sem obra · número repetido na empresa · visualizador recusado · comercial cria · limpeza dos 3 contratos avulsos.

Os 5 passos do navegador: o botão da listagem abre o form · envio vazio mostra "Número obrigatório" e "Selecione uma obra" sem sair da tela · desconto acima do valor e soma de 110% são barrados com a mensagem de cada um · nada foi gravado enquanto o form estava inválido · corrigido, cria o contrato ativo e sem origem, com os percentuais convertidos para fração. Screenshot: `21-contrato-avulso-erros.png`.

Conferido em gc-dev depois da rodada: 0 contratos e 0 propostas `RUN-` / `VALIDA-` sobrando e 0 itens órfãos.

4. O que não foi validado

- **Obra de outra empresa no corpo da requisição:** a FK composta `contratos_obra_fk` (obra + empresa) recusa, e o erro é traduzido para "Obra inválida para esta empresa". Mas gc-dev tem uma empresa só, então nenhum passo mandou uma obra alheia.
- **A prévia do valor final com desconto inválido:** enquanto o desconto passa do valor, a caixa "Valor final" mostra um número negativo (-R\$ 1.000,00 no screenshot), com o erro logo acima. É o mesmo comportamento do formulário de proposta, herdado de `calcularValorFinal`. Não mudei porque ele é compartilhado com a proposta; fica registrado.
- **A nota "Anexos podem ser gerenciados na aba Anexos da tela de detalhes":** a aba é do 6.4 e do 6.6. Até lá, o texto promete uma tela que não existe.
- **`propostaToContratoFormValues` sem teste unitário:** continua a pendência do 6.2.
- **Destino depois de criar:** é a listagem filtrada até o 6.4.

5. Registro para quem for operar gc-prod

Nada novo. O 6.3 não tem migration. As três migrations da sprint estão no documento do 6.1, seção 6.